

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0515

Slot

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool Weightage: 40%

QUESTIONS:3

Total Marks 30

Duration :60 Minutes

Annexure A

Experiments

Code of Conduct:

I P.Paneendra (192321072) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

p.paneendra

Signature of the student:

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

1. Harris Corner Detection Analysis

Perform Harris corner detection on images containing varying textures and patterns. Explain the concept of corner detection and its importance in computer vision tasks. Use appropriate parameters and tools to execute the experiment, document the detected corner points clearly, and analyze the distribution, accuracy, and significance of the detected corners.

Aim

To perform Harris Corner Detection on images with varying textures and patterns, to understand the concept and importance of corner detection in computer vision, and to analyze the distribution and significance of detected corners.

Algorithm (Step-by-Step)

1. Read the input image and convert it to grayscale.
2. Compute image gradients I_x and I_y using a Sobel operator.
3. Compute products of derivatives: I_x^2 , I_y^2 , $I_x I_y$.
4. Apply Gaussian smoothing to these products to get the elements of matrix M at each pixel.
5. Compute the corner response $R = \det(M) - k(\text{trace}(M))^2$ for every pixel.
6. Threshold R to retain only strong corner responses.
7. Apply non-maximum suppression to keep only local maxima (avoid clustered detections).
8. Mark/overlay detected corner points on the original image.
9. Analyze the spatial distribution of corners with respect to image texture/pattern.

Program (Python):

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

# Step 1: Load image

img = cv2.imread('input_texture.jpg')

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

gray_f = np.float32(gray)

# Step 2-6: Harris Corner Detection (OpenCV built-in implements steps 2-6)
```

```

# blockSize: neighborhood size, ksize: Sobel aperture, k: Harris free parameter

dst = cv2.cornerHarris(gray_f, blockSize=2, ksize=3, k=0.04)


# Dilate to mark corners clearly

dst = cv2.dilate(dst, None)


# Step 7: Threshold for strong corners (non-max suppression approximated via thresholding)

threshold = 0.01 * dst.max()

corner_img = img.copy()

corner_img[dst > threshold] = [0, 0, 255] # mark corners in red


# Count and log corner coordinates

ys, xs = np.where(dst > threshold)

print(f"Number of corner points detected: {len(xs)}")

corner_coords = list(zip(xs, ys))

print("Sample corner coordinates:", corner_coords[:10])


# Step 8: Display result

plt.figure(figsize= (10,5))

plt.subplot(1,2,1); plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)); plt.title('Original'); plt.axis('off')

plt.subplot(1,2,2); plt.imshow(cv2.cvtColor(corner_img, cv2.COLOR_BGR2RGB)); plt.title('Harris
Corners'); plt.axis('off')

plt.tight_layout()

plt.savefig('harris_output.png', dpi=150)

plt.show()

```

Output (Documentation Template)

Image Type	Corners Detected	Observation
Chessboard (high texture, regular pattern)	High count, evenly spaced	Corners align precisely with grid intersections
Building / architecture (structured edges)	Moderate count	Corners concentrated at window edges, roof lines
Natural scene (grass, sky — low texture)	Very few / sparse	Flat regions produce almost no corners, as expected
Textured cloth / carpet pattern	High count, dense clusters	Corners cluster densely wherever local pattern repeats

2. Effect of Noise on Corner Detection

Design an experiment to study the effect of noise on Harris corner detection performance. Explain the purpose of introducing noise and its influence on image features. Add noise systematically, perform corner detection using suitable tools, document observations for original and noisy images, and analyze the impact of noise along with possible improvement techniques.

Aim

To design and conduct an experiment studying the effect of noise on the performance of Harris corner detection, and to analyze how noise degrades feature detection along with possible mitigation techniques.

Algorithm (Step-by-Step)

1. Load the original clean image and convert to grayscale.
2. Apply Harris corner detection on the original image (baseline).
3. Systematically add noise (Gaussian with increasing variance: $\sigma = 5, 15, 30$; salt-and-pepper with increasing density) to generate multiple noisy versions.
4. Apply Harris corner detection identically (same parameters) on each noisy version.
5. Count and visualize detected corners for each noise level.
6. Compare corner count, position shift, and false-positive rate against the baseline.
7. Apply a denoising filter (Gaussian blur / median filter) to noisy images and repeat detection to evaluate improvement.

8. Tabulate and plot corner count vs. noise level.

Program (Python + OpenCV):

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

def add_gaussian_noise(img, sigma):

    noise = np.random.normal(0, sigma, img.shape).astype(np.float32)

    noisy = img.astype(np.float32) + noise

    return np.clip(noisy, 0, 255).astype(np.uint8)

def add_salt_pepper(img, amount=0.02):

    noisy = img.copy()

    num_salt = int(amount * img.size * 0.5)

    num_pepper = int(amount * img.size * 0.5)

    coords = [np.random.randint(0, i, num_salt) for i in img.shape]

    noisy[tuple(coords)] = 255

    coords = [np.random.randint(0, i, num_pepper) for i in img.shape]

    noisy[tuple(coords)] = 0

    return noisy

def harris_count(gray, k=0.04, thresh_ratio=0.01):

    gray_f = np.float32(gray)

    dst = cv2.cornerHarris(gray_f, 2, 3, k)

    dst = cv2.dilate(dst, None)

    thresh = thresh_ratio * dst.max()
```

```

    return np.sum(dst > thresh), dst, thresh

img = cv2.imread('input_texture.jpg', cv2.IMREAD_GRAYSCALE)

# Baseline
base_count, _, _ = harris_count(img)

# Systematic noise addition
sigmas = [5, 15, 30]
results = {'Original': base_count}

for s in sigmas:

    noisy = add_gaussian_noise(img, s)

    cnt, _, _ = harris_count(noisy)

    results[f'Gaussian  $\sigma$ ={s}'] = cnt

sp_noisy = add_salt_pepper(img, amount=0.03)
results['Salt & Pepper (3%)'] = harris_count(sp_noisy)[0]

# Denoising + re-detection (improvement technique)
denoised = cv2.GaussianBlur(add_gaussian_noise(img, 30), (5,5), 0)
results['Denoised ( $\sigma$ =30 + Gaussian Blur)'] = harris_count(denoised)[0]

for k, v in results.items():

    print(f"{k}: {v} corners")

# Plot

```

```
plt.bar(results.keys(), results.values())

plt.xticks(rotation=45, ha='right')

plt.ylabel('Number of Corners Detected')

plt.title('Effect of Noise on Harris Corner Count')

plt.tight_layout()

plt.savefig('noise_effect.png', dpi=150)

plt.show()
```

Output (Documentation Template)

Condition	Corner Count	Observation
Original (no noise)	Baseline (e.g., 480)	Clean, well-localized corners
Gaussian noise $\sigma=5$	Slightly higher/similar	Minor spurious corners begin to appear
Gaussian noise $\sigma=15$	Noticeably higher	Many false corners in flat regions
Gaussian noise $\sigma=30$	Sharp increase or scattered drop (parameter-dependent)	Severe degradation, gradients dominated by noise
Salt & pepper (3%)	High spike in false corners	Impulse noise creates isolated high-gradient points
Denoised (blur applied)	Closer to baseline	Filtering restores structure, reduces false positives

3. Hough Transform for Shape Detection

Conduct an experiment using the Hough Transform to detect lines or circles in an image. Explain the principles and significance of the Hough Transform in shape detection. Execute the detection process using appropriate tools, record the detected shapes and their parameters, and analyze the detection accuracy under different image conditions.

Aim

To detect lines and/or circles in an image using the Hough Transform, understand its underlying principles, and analyze detection accuracy under different image conditions.

Algorithm (Step-by-Step)

1. Read the input image and convert to grayscale.
2. Apply Gaussian blur to reduce noise.
3. Perform edge detection (Canny) to obtain a binary edge map.
4. For lines: Apply `cv2.HoughLines` (standard) or `cv2.HoughLinesP` (probabilistic) on the edge map.
5. For circles: Apply `cv2.HoughCircles` (Hough Gradient method) directly on the blurred grayscale image.
6. Extract parameters of each detected shape: (ρ, θ) or endpoints for lines; (a, b, r) for circles.
7. Draw detected lines/circles on a copy of the original image.
8. Repeat detection under varying conditions (different Canny thresholds, added noise, varying `HoughCircles` parameters) to test robustness.
9. Record and compare accuracy (correct detections vs. missed/false detections) across conditions.

Program (Python + OpenCV)

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img = cv2.imread('shapes_input.jpg')

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

blurred = cv2.GaussianBlur(gray, (5, 5), 0)
```



```
# ---- Line Detection ----
```

```
edges = cv2.Canny(blurred, 50, 150)
```

```
lines = cv2.HoughLinesP(edges, rho=1, theta=np.pi/180, threshold=80,  
                        minLineLength=50, maxLineGap=10)
```

```
line_img = img.copy()
```

```
if lines is not None:
```

```
    for l in lines:
```

```
        x1, y1, x2, y2 = l[0]
```

```
        cv2.line(line_img, (x1, y1), (x2, y2), (0, 255, 0), 2)
```

```
    print(f"Number of line segments detected: {len(lines)}")
```

```
# ---- Circle Detection ----
```

```
circle_img = img.copy()
```

```
circles = cv2.HoughCircles(blurred, cv2.HOUGH_GRADIENT, dp=1, minDist=30,  
                          param1=100, param2=40, minRadius=10, maxRadius=100)
```

```
if circles is not None:
```

```
    circles = np.uint16(np.around(circles))
```

```
    print(f"Number of circles detected: {len(circles[0])}")
```

```
    for c in circles[0, :]:
```

```
        cv2.circle(circle_img, (c[0], c[1]), c[2], (255, 0, 0), 2) # circle outline
```

```
        cv2.circle(circle_img, (c[0], c[1]), 2, (0, 0, 255), 3) # center point
```

```
    print(f"Center=({c[0]},{c[1]}), Radius={c[2]}")
```

```
# Display
```

```
plt.figure(figsize=(12,4))

plt.subplot(1,3,1); plt.imshow(edges, cmap='gray'); plt.title('Canny Edges'); plt.axis('off')

plt.subplot(1,3,2); plt.imshow(cv2.cvtColor(line_img, cv2.COLOR_BGR2RGB)); plt.title('Detected Lines');
plt.axis('off')

plt.subplot(1,3,3); plt.imshow(cv2.cvtColor(circle_img, cv2.COLOR_BGR2RGB)); plt.title('Detected
Circles'); plt.axis('off')

plt.tight_layout()

plt.savefig('hough_output.png', dpi=150)

plt.show()
```

Output (Documentation Template)

Shape Type	Test Condition	Detected Parameters	Accuracy Observation
Lines	Clean image, sharp edges	e.g., 12 line segments, correct slope/position	High accuracy, matches visible edges
Lines	Low Canny threshold	Excess spurious short segments	Over-detection / noise sensitivity
Lines	High Canny threshold	Missed faint lines	Under-detection
Circles	Well-lit, distinct circular objects (coins, wheels)	Center (a,b) and radius r close to ground truth	High accuracy
Circles	Overlapping/occluded circles	Some circles missed or center slightly off	Reduced accuracy due to gradient interference
Circles	Noisy image	False circles or missed true circles	param2 (accumulator threshold) needs tuning